

Lab 8

Deep learning 1 - Recurrent Neural Networks

Learning objectives for this week are:

1. how to build a model using Tensorflow and Keras
2. to understand how Recurrent Neural Networks work
3. to be able to assess their performance and expressiveness

8.1 Getting started

While machine learning uses simpler concepts, **deep learning** (a subset of machine learning) works with **artificial neural networks** (ANN), which are designed to imitate how humans think and learn. The term ‘**deep learning**’ is coined because the neural networks have various layers that enable learning, and re-learning.

In the same way that the human brain is made up of neurons, neural networks are built up of layers of nodes. Individual layer nodes are linked to nodes in neighboring layers. The number of layers in the network indicates how deeper it is.

In the human brain, a single neuron gets hundreds of impulses from other neurons. In an artificial neural network, signals move between nodes by applying weights to them. A heavier weighted node will have a greater impact on the next layer of nodes. The weighted inputs are compiled in the last layer to produce an output.

Because deep learning systems process a vast quantity of data and perform multiple difficult mathematical calculations, they demand strong hardware. Deep learning training computations, even with such sophisticated technology, can take a long time.

Widely-adopted deep learning frameworks include:

- PyTorch (highly flexible/research),
- TensorFlow (production-ready/scalable),
- Keras (beginner-friendly API).
- JAX (high-performance).

8.2 Installing TensorFlow and Keras

Install Tensorflow:

```
pip install --upgrade tensorflow
```

If you want to verify the installation:

```
python -c "import tensorflow as tf; print(tf.reduce_sum(tf.random.normal([1000, 1000])))"
```

Install Keras:

```
pip install --upgrade keras
```

TensorFlow - is an open-source deep-learning library, developed by Google, and used for numerical computation, large-scale machine learning, deep learning, and other statistical and predictive analytics workloads, originally designed for production and scalability. It's developed in C++ and has its implementation in Python. Keras can now be run on top of TensorFlow.

Google's TensorFlow is currently the most popular learning library in the world. It's based on the concept of tensors, which are vectors or matrices of n dimensions.

PyTorch is an open source deep learning framework built to be **flexible** and modular for **research**, with the stability and support needed for production deployment.

JAX is a Python library for accelerator-oriented array computation and program transformation, designed for high-performance numerical computing and large-scale machine learning. It is focused on performance + functional programming.

Keras is a high-level, open-source deep learning API written in Python, designed to be user-friendly, modular, and extensible. As of Keras 3, it is a multi-backend framework that allows users to run workflows on top of **JAX, TensorFlow, PyTorch**. Keras provides a complete framework to create any type of neural networks. It supports simple neural network to very large and complex neural network model. Let's understand the architecture of Keras framework and how Keras helps in deep learning.

Architecture of Keras. Keras API can be divided into 2 main categories: **Models** and **Layers**. You can read more about Keras here .

8.2.1 Keras Models

In Keras, every ANN is represented by **Keras Models**. Every Keras Model is composition of **Keras Layers** and represents ANN layers like input, hidden layer, output layers, convolution layer, pooling layer, etc., Keras model and layers access Keras modules for activation function, loss function, regularization function, etc.,

- **Sequential model** - the simplest type of model. Read about sequential model.
- **Functional API** is basically used to create complex models. Functional API

A Sequential model is appropriate for a plain stack of layers where each layer has exactly one input tensor and one output tensor.

You can create a Sequential model by passing a list of layers to the Sequential constructor:

```
1  """import tensorflow as tf
2  from tensorflow import keras
3  from tensorflow.keras import layers
4
5  model = keras.Sequential(
6      [
7          layers.Dense(2, activation="relu"),
```

```

8     layers.Dense(3, activation="relu"),
9     layers.Dense(4),
10 ]
11 )

```

You can also create a Sequential model incrementally via the `add()` method:

```

1  "model = keras.Sequential()
2  model.add(layers.Dense(2, activation="relu"))
3  model.add(layers.Dense(3, activation="relu"))
4  model.add(layers.Dense(4))

```

However, it can be very useful when building a Sequential model incrementally to be able to display the summary of the model so far, including the current output shape. In this case, you should start your model by passing an `Input` object to your model, so that it knows its input shape from the start:

```

1  "model = keras.Sequential()
2  model.add(keras.Input(shape=(4,)))
3  model.add(layers.Dense(2, activation="relu"))
4
5  model.summary()
6
7  #Note that the Input object is not displayed as part of model.layers, since it isn't a layer:
8
9  model.layers

```

A simple alternative is to just pass an `input_shape` argument to your first layer:

```

1  "model = keras.Sequential()
2  model.add(layers.Dense(2, activation="relu", input_shape=(4,)))
3
4  model.summary()

```

With the "Functional API", we can create **Functional models**, where you start from `Input`, you chain layer calls to specify the model's forward pass, and finally you create your model from inputs and outputs:

```

1  "import tensorflow as tf
2  from keras.models import Model
3  from keras.layers import Dense, Input
4
5  inputs = Input(shape=(3,)) #input layer
6  x = Dense(4, activation=tf.nn.relu)(inputs) #hidden layer
7  outputs = Dense(5, activation=tf.nn.softmax)(x) #output layer
8  model = Model(inputs=inputs, outputs=outputs)

```

The following **Multilayer Perceptron** model has 10 inputs, 3 hidden layers with 10, 20, and 10 neurons, and an output layer with 1 output. Rectified linear activation functions are used in each hidden layer and a sigmoid activation function is used in the output layer, for binary classification.

```

1  "## Multilayer Perceptron
2  from keras.utils import plot_model
3  from keras.models import Model
4  from keras.layers import Input
5  from keras.layers import Dense
6
7  visible = Input(shape=(10,))
8  hidden1 = Dense(10, activation='relu')(visible)
9  hidden2 = Dense(20, activation='relu')(hidden1)
10 hidden3 = Dense(10, activation='relu')(hidden2)
11 output = Dense(1, activation='sigmoid')(hidden3)
12 model = Model(inputs=visible, outputs=output)
13
14 # summarize layers
15 print(model.summary())
16 # plot graph
17 plot_model(model, to_file='multilayer_perceptron_graph.png')

```

8.2.2 Keras Layers

Each Keras layer in the Keras model represent the corresponding layer (input layer, hidden layer and output layer) in the actual proposed neural network model. Keras provides a lot of pre-build layers so that any complex neural network can be easily created.

Some of the important Keras layers are: the base Layer class, core layers, layer weight initializers, layer weight regularizers, Convolution Layers, Pooling Layers, Recurrent Layers and other layers like Preprocessing layers, Normalization layers, Regularization layers, Attention layers, Reshaping layers, Merging layers, Locally-connected layers, Activation layers, etc.

Each layer receives input information, do some computation and finally output the transformed information. The output of one layer will flow into the next layer as its input.

Keras layer requires below minimum details to create a complete layer:

- Shape of the input data - all type of input data like text, images or videos will be first converted into array of numbers and then feed into the algorithm.

- Number of neurons / units in the layer

- Initializers provide different functions to set these initial weight.

- Regularizers provide functions to set penalties on the layer.

- Constraints provide different functions to set the constraint on the layer.

- Activations are special functions used to find whether a specific neuron is activated or not.

You can read more about layers here. Some of the most used layers:

Dense layer is the regular deeply connected neural network layer. It is most common and frequently used layer. Dense layer does the below operation on the input and return the output: $\text{output} = \text{activation}(\text{dot}(\text{input}, \text{kernel}) + \text{bias})$

```
1  """>>> from keras.models import Sequential
2  >>> from keras.layers import Activation, Dense
3
4  >>> model = Sequential()
5  # number of units = 16
6  >>> layer_1 = Dense(16, input_shape = (8,))
7  >>> model.add(layer_1)
8  >>> layer_1.input_shape
9  (None, 8)
10 >>> layer_1.output_shape
11 (None, 16)
12 >>> layer_1.get_weights()
13 #returns the full list of the weights used in the layer.
```

Dropout layer is a regularization layer and is used to fix the over-fitting issue. Input data may have some of the unwanted data, usually called as Noise. Dropout will try to remove the noise data and thus prevent the model from over-fitting.

Flatten layer is a reshaping layer, that flattens the input. Does not affect the batch size.

TextVectorization layer is a preprocessing layer which maps text features to integer sequences. This layer has basic options for managing text in a Keras model. It transforms a batch of strings (one example = one string) into either a list of token indices (one example = 1D tensor of integer token indices)

Conv2D is a convolution 2D layer. It creates a convolutional kernel with the layer input to produce a tensor of outputs. If use_bias is True, a bias vector is created and added to the outputs. Finally, if activation is not None, it is applied to the outputs as well.

When using this layer as the first layer in a model, provide the keyword argument input_shape (tuple of integers, does not include the sample axis), e.g. input_shape=(128, 128, 3) for 128x128 RGB pictures in data_format="channels.last".

LSTM Layer - Long Short-Term Memory layer - is a recurrent layer where the output is to be fed back to input. This layer will choose different implementations (cuDNN-based or pure-TensorFlow) to maximize the performance, based on available runtime hardware and

constraints. It has a "recurrent_activation" parameter that uses an activation function for the recurrent step.

cuDNN (The NVIDIA CUDA Deep Neural Network library) is a GPU-accelerated library of primitives for deep neural networks. cuDNN provides highly tuned implementations for standard routines such as forward and backward convolution, pooling, normalization, and activation layers.

Embedding layer turns positive integers (indexes) into dense vectors of fixed size.

e.g. $[[4], [20]] \implies [[0.25, 0.1], [0.6, -0.2]]$

This layer can only be used as the first layer in a model. This is mainly used in Natural Language Processing related applications such as language modeling, but it can also be used with other tasks that involve neural networks. While dealing with NLP problems, we can use pre-trained word embeddings such as GloVe. Alternatively we can also train our own embeddings using Keras embedding layer.

Embedding layer enables us to convert each word into a fixed length vector of defined size. The resultant vector is a dense one with having real values instead of just 0's and 1's. The fixed length of word vectors helps us to represent words in a better way along with reduced dimensions.

Keras also provides a lot of built-in neural network related functions to properly create the Keras model and Keras layers. Some of the functions are:

- Model training APIs here : compile, fit, evaluate, predict methods, etc.
- Activations module - activation functions like softmax, relu, sigmoid etc. Check here
- Loss module - loss functions like *mean_squared_error*, *mean_absolute_error*, etc., here
- Optimizer module - optimizer functions like adam, sgd, etc., Available optimizers
- Regularizers - regularization functions like L1 regularizer, L2 regularizer, etc. Check here
- Metrics - used to judge the performance of your model. Check here.

Build a Model in Keras

Basic steps involved in building a model in Keras link:

1. Define a network: You define the different layers in our model and the connections between them. Keras has two main types of models: **Sequential** and **Functional models**. You choose which type of model you want and then define the dataflow between them.

2. Compile a network: In Keras, the `model.compile()` method performs this function. To compile the model, we define the **loss function** which calculates the losses in our model, the **optimizer** which reduces the loss, and the **metrics** which is used to find the accuracy of our model.

3. Fit the network: Using this, we fit our model to our data after compiling. This is used to train the model on our data. We call `fit()`, which will train the model by slicing the data into "batches" of size *batch_size*, and repeatedly iterating over the entire dataset for a given number of epochs.

4. Evaluate the network: After fitting our model, we need to evaluate the error in our model.

5. Make Predictions: We use `model.predict()` to make predictions using our model on new data.

Exercise 8.1 Follow the tutorial with TensorFlow: TensorFlow 2 quickstart for beginners.

Exercise 8.2 Follow the tutorial for classification: Basic classification: Classify images of clothing.

Exercise 8.3 Follow the example for Introduction to Deep Learning with Keras . Download the notebook file and try to run on your side.

8.3 Recurrent Neural Networks

Some useful links:

The Keras Functional API

Recurrent layers with Keras.

Recurrent Neural Networks (RNNs) are a class of neural networks designed to process sequential data by using internal memory to remember previous inputs (where order matters). They are used in various applications, especially in natural language processing (sentences, text, language modeling) and time series (stock prices, weather) analysis. Unlike feedforward networks, RNNs have memory: they pass hidden states from one step to the next.

Types of RNN Layers (Keras)

Keras provides several RNN variants:

Layer	Strength	Weakness	Use Case
SimpleRNN	Easy, fast	Vanishing gradient problem	Short sequences
LSTM	Long-term memory	Slower, more parameters	Long dependencies
GRU	Faster than LSTM	Slightly less expressive	Faster alternative
Bidirectional	Uses past + future context	Doubles computation	Context both directions

Table 8.1: Comparison of RNN Layer Types

- Long Short-Term Memory (LSTM) networks: Use "gates" to control the flow of information, allowing them to remember context over much longer sequences.
- Gated Recurrent Units (GRU): A simpler, more efficient variant of LSTMs that also addresses the vanishing gradient problem.
- Bidirectional LSTM (BiLSTM) : a layer that processes a sequence in both forward and backward directions, allowing it to use information from past and future time steps simultaneously. This improves performance on tasks where context matters, such as natural language processing.

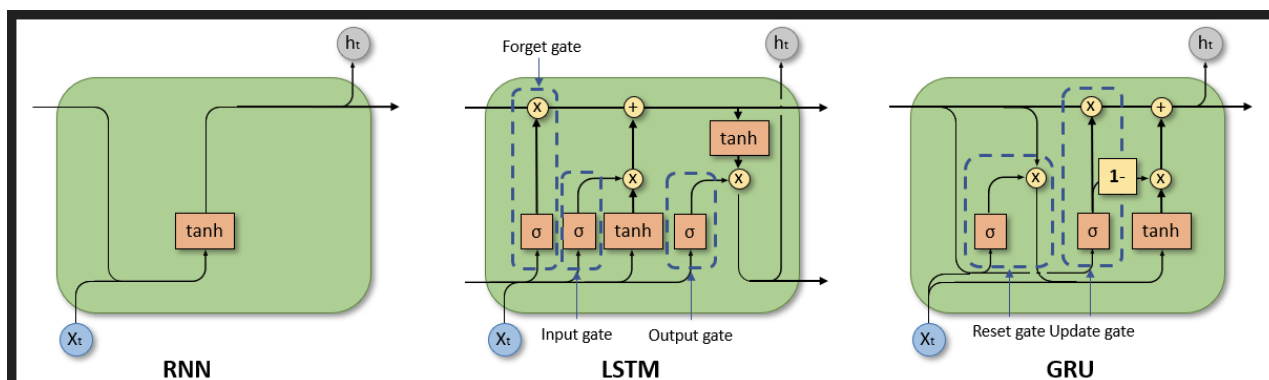


Figure 8.1: RNN, LSTM, and GRU cells

RNNs can be categorized by how they handle inputs and outputs (sequence length), which dictates their use cases.

Type of RNN	Input	Output	Example Applications
One-to-Many	Single input	Sequence of outputs	Image captioning, where a single image generates a descriptive sentence; music generation, where a starting note generates a composition.
Many-to-One	Sequence of inputs	Single output	Sentiment analysis, classifying a movie review as positive or negative based on the text sequence; action recognition, identifying a single action from a sequence of video frames.
Many-to-One	Sequence of inputs	Single output	Sentiment analysis, classifying a movie review as positive or negative based on the text sequence; action recognition, identifying a single action from a sequence of video frames.
Many-to-Many	Sequence of inputs	Sequence of outputs	Machine translation, translating an English sentence into French; video captioning, generating a description for a sequence of video frames; text summarization, condensing a long document into a shorter one.

Table 8.2: Types of RNN systems

Specific Use Cases

- Speech recognition and synthesis: converting spoken words into text or generating speech from text.
- Time Series prediction: forecasting future values based on historical data, such as stock prices or weather conditions.
- Handwriting recognition: interpreting a sequence of pen strokes to recognize handwritten text.

Exercise 8.4 *Follow the example. Create a new Jupyter Notebook and write the code to :*

1. *Load a dataset(E.g. tensorflow.keras.datasets.mnist), split data into train and test, normalize the data (you can use the preprocessing layers)*
2. *Define the LSTM/CuDNNLSTM model(define the layers of the model-input layers + hidden layers+output layer)*
3. *Compile the model (using a loss function that you want to optimize + an algorithm to perform the optimization procedure + performance metrics)*
4. *Fit the model (specify number of epochs (loops through the training dataset) and the batch size (number of samples in an epoch))*
5. *Evaluate the model(with data which is not used in the training process)*
6. *Make predictions*

More examples on Recurrent Neural Networks - RNN: TensorFlow example and Recurrent Neural Networks Example

Bibliography

1. Keras tutorial https://keras.io/getting_started/
2. Keras Guide <https://keras.io/guides/>
3. RNN - <https://www.ibm.com/think/topics/recurrent-neural-networks>
4. RNN - <https://stanford.edu/~shervine/teaching/cs-230/cheatsheet-recurrent-neural-networks/>